# LeetCode

## Top Interview Questions

### Medium & Hard · FAANG + Top Tech Companies · 2025

50 Most Frequently Asked Problems with Approaches, Complexity Analysis & Java Solutions

Amazon · Google · Meta · Microsoft · Apple · Netflix · Uber · LinkedIn · Twitter

# Problem Index — 50 Questions by Category

| # | Problem | Difficulty | Category | Top Companies |
|---|---|---|---|---|
| 1 | Two Sum | Medium | Array / HashMap | Amazon, Google, Meta |
| 2 | Longest Substring Without Repeating Characters | Medium | Sliding Window | Amazon, Google, Microsoft |
| 3 | 3Sum | Medium | Two Pointers | Amazon, Google, Adobe |
| 4 | Container With Most Water | Medium | Two Pointers | Amazon, Google |
| 5 | Longest Palindromic Substring | Medium | DP / Expand Center | Amazon, Microsoft, Apple |
| 6 | Group Anagrams | Medium | HashMap | Amazon, Google, Microsoft |
| 7 | Word Search | Medium | DFS/Backtracking | Amazon, Microsoft, Google |
| 8 | Product of Array Except Self | Medium | Prefix/Suffix | Amazon, Microsoft, Apple |
| 9 | Find Minimum in Rotated Sorted Array | Medium | Binary Search | Microsoft, Amazon, Meta |
| 10 | Search in Rotated Sorted Array | Medium | Binary Search | Amazon, Google, Meta |
| 11 | Merge Intervals | Medium | Sorting + Sweep | Google, Facebook, Amazon |
| 12 | Jump Game II | Medium | Greedy | Amazon, Google |
| 13 | Spiral Matrix | Medium | Simulation | Amazon, Microsoft, Google |
| 14 | Rotate Image | Medium | In-place Matrix | Amazon, Microsoft |
| 15 | Coin Change | Medium | DP | Amazon, Google, Microsoft |
| 16 | Longest Increasing Subsequence | Medium | DP / Binary Search | Amazon, Google, Uber |
| 17 | Decode Ways | Medium | DP | Meta, Amazon, Google |
| 18 | Unique Paths | Medium | DP | Amazon, Microsoft, Google |
| 19 | Number of Islands | Medium | BFS/DFS | Amazon, Google, Microsoft |
| 20 | Clone Graph | Medium | BFS/DFS | Meta, Amazon, Google |
| 21 | Pacific Atlantic Water Flow | Medium | BFS/DFS | Google, Amazon |
| 22 | Course Schedule | Medium | Topological Sort | Amazon, Google, Meta |
| 23 | Reorder List | Medium | Linked List | Amazon, Google, Microsoft |
| 24 | LRU Cache | Medium | HashMap + DLL | Amazon, Google, Microsoft |
| 25 | Binary Tree Level Order Traversal | Medium | BFS | Amazon, Google, Meta |
| 26 | Validate Binary Search Tree | Medium | DFS/Inorder | Amazon, Microsoft, Google |

| # | Problem | Difficulty | Category | Top Companies |
|---|---|---|---|---|
| 27 | Kth Smallest Element in BST | Medium | DFS Inorder | Google, Amazon, LinkedIn |
| 28 | Construct Binary Tree from Preorder+Inorder | Medium | Recursion | Amazon, Microsoft |
| 29 | Word Ladder | Hard | BFS | Amazon, Google, LinkedIn |
| 30 | Serialize & Deserialize Binary Tree | Hard | BFS/DFS | Google, Amazon, Meta |
| 31 | Trapping Rain Water | Hard | Two Pointers / Stack | Amazon, Google, Meta |
| 32 | Median of Two Sorted Arrays | Hard | Binary Search | Google, Amazon, Microsoft |
| 33 | Merge K Sorted Lists | Hard | Heap | Amazon, Google, Meta |
| 34 | Word Break II | Hard | DP + Backtracking | Amazon, Google |
| 35 | N-Queens | Hard | Backtracking | Amazon, Google, Microsoft |
| 36 | Largest Rectangle in Histogram | Hard | Stack | Amazon, Google |
| 37 | Sliding Window Maximum | Hard | Deque | Amazon, Google |
| 38 | Minimum Window Substring | Hard | Sliding Window | Amazon, Google, LinkedIn |
| 39 | Regular Expression Matching | Hard | DP | Google, Meta, Amazon |
| 40 | Find Median from Data Stream | Hard | Two Heaps | Amazon, Google |
| 41 | Alien Dictionary | Hard | Topological Sort | Google, Amazon, Meta |
| 42 | Edit Distance | Hard | DP | Amazon, Google, Microsoft |
| 43 | Palindrome Partitioning II | Hard | DP | Amazon, Google |
| 44 | Maximum Profit in Job Scheduling | Hard | DP + Binary Search | Google, Amazon |
| 45 | Reverse Nodes in K-Group | Hard | Linked List | Amazon, Google, Meta |
| 46 | Basic Calculator II | Medium | Stack | Amazon, Google |
| 47 | Subsets II | Medium | Backtracking | Amazon, Google, Microsoft |
| 48 | Top K Frequent Elements | Medium | Heap / Bucket Sort | Amazon, Google, Meta |
| 49 | Meeting Rooms II | Medium | Sweep Line / Heap | Amazon, Google, Meta |
| 50 | Task Scheduler | Medium | Greedy / Counting | Amazon, Google |

# 📊 Category I — Arrays, Two Pointers & Sliding Window

LeetCode's most common interview pattern

## #1 Two Sum [Medium]

**Companies:** Amazon, Google, Meta, Apple, Microsoft

### Approach & Key Insight

**Problem: Given array nums and target, return indices of two numbers that add up to target.**

**Approach: HashMap (One-Pass)**

**- For each number nums[i], compute complement = target - nums[i]**

**- Check if complement already exists in the HashMap**

**- If yes → return [map.get(complement), i]**

**- If no → store nums[i] → i in the HashMap**

**- One pass: by the time we need the pair, the first element is already stored**

**Why HashMap: Reduces O(n²) brute-force to O(n) by trading space for lookup speed.**

| ⏱ **Time:** O(n) | 💾 **Space:** O(n) |
|---|---|

### Java Solution

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        // num → index mapping
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{}; // guaranteed to have solution
    }
}
```

## #2 Longest Substring Without Repeating Characters [Medium]

**Companies:** Amazon, Google, Microsoft, Adobe

### Approach & Key Insight

**Problem: Find length of longest substring with all unique characters.**

**Approach: Sliding Window with HashMap**

**- Maintain a window [left, right] where all characters are unique**

**- Expand right pointer: add chars[right] to a HashMap (char → index)**

**- When duplicate found: move left pointer to max(left, lastSeen[char] + 1)**

**(not just left+1, because the duplicate might be far left of current window)**

**- Track maxLen = max(maxLen, right - left + 1) at each step**

**Key insight: Store the LAST SEEN INDEX of each char to jump left pointer efficiently.**

⏱ **Time:** O(n)

💾 **Space:** O(min(m,n)) — m=charset size

### Java Solution

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>(); // char → last seen index
        int maxLen = 0, left = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            // If char seen within current window, shrink from left
            if (map.containsKey(c) && map.get(c) >= left) {
                left = map.get(c) + 1;
            }
            map.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

## #3 3Sum [Medium]

**Companies:** Amazon, Google, Adobe, Microsoft

### Approach & Key Insight

**Problem: Find all unique triplets in array that sum to zero.**

**Approach: Sort + Two Pointers**

**- Sort the array first (enables two-pointer and easy deduplication)**

**- For each index i, use two pointers (lo=i+1, hi=n-1) to find pairs summing to -nums[i]**

**- If sum < 0: move lo right (need larger value)**

**- If sum > 0: move hi left (need smaller value)**

**- If sum == 0: record result; skip duplicates by advancing lo/hi past same values**

**- Skip duplicate values at i (if nums[i] == nums[i-1], continue)**

**Deduplication is the tricky part: must skip duplicates at all three pointer positions.**

| | |
|---|---|
| ⏱ **Time:** $O(n^2)$ | 💾 **Space:** O(1) excluding output |

### Java Solution

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            // Skip duplicate values for i
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            // Early termination: smallest possible sum > 0
            if (nums[i] > 0) break;

            int lo = i + 1, hi = nums.length - 1;
            while (lo < hi) {
                int sum = nums[i] + nums[lo] + nums[hi];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                    // Skip duplicates at lo and hi
                    while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                    while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                    lo++; hi--;
                } else if (sum < 0) {
                    lo++;
                } else {
                    hi--;
                }
            }
        }
        return result;
    }
}
```

## #4 Container With Most Water [Medium]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Given heights array, find two lines that form container with most water.**

**Approach: Two Pointers (Greedy)**

**- Start with widest possible container: left=0, right=n-1**

**- Water = min(height[left], height[right]) * (right - left)**

**- Which pointer to move? Moving the TALLER side can only decrease width without**

**increasing height → never helps. Always move the SHORTER side inward.**

**- Moving shorter side might find a taller line that compensates for lost width.**

**Proof: If we don't move the shorter side, no better solution is possible on that side.**

| ⏱ **Time:** O(n) | 💾 **Space:** O(1) |
|---|---|

### Java Solution

```
class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1;
        int maxWater = 0;

        while (left < right) {
            int h = Math.min(height[left], height[right]);
            int water = h * (right - left);
            maxWater = Math.max(maxWater, water);

            // Move the shorter side — moving taller can't improve result
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxWater;
    }
}
```

## #5 Minimum Window Substring [Hard]

**Companies:** Amazon, Google, LinkedIn, Meta

### Approach & Key Insight

**Problem: Find minimum window in s that contains all chars of t.**

**Approach: Sliding Window with Two HashMaps**

**- Use need map (chars needed from t) and have map (chars in current window)**

**- formed = number of unique chars in window satisfying required frequency**

**- Expand right: add char, update have map, check if this char now satisfies need**

**- When formed == required: try to shrink from left to find smaller valid window**

**- Shrink left: remove leftmost char, update have/formed, record answer**

**Key insight: Track how many unique characters currently meet their required frequency.**

**This avoids checking the entire map on each step → O(n) total.**

| ⏱ **Time:** O(\|s\| + \|t\|) | 💾 **Space:** O(\|s\| + \|t\|) |
| --- | --- |

## Java Solution

```
class Solution {
    public String minWindow(String s, String t) {
        if (s.isEmpty() || t.isEmpty()) return "";

        Map<Character,Integer> need = new HashMap<>(), have = new HashMap<>();
        for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

        int required = need.size(), formed = 0;
        int left = 0, minLen = Integer.MAX_VALUE, minLeft = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            have.merge(c, 1, Integer::sum);
            // Check if this char's count now meets requirement
            if (need.containsKey(c) && have.get(c).equals(need.get(c))) formed++;

            // Shrink window while valid
            while (formed == required) {
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    minLeft = left;
                }
                char lc = s.charAt(left++);
                have.merge(lc, -1, Integer::sum);
                if (need.containsKey(lc) && have.get(lc) < need.get(lc)) formed--;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(minLeft, minLeft +
minLen);
    }
}
```

## #6 Trapping Rain Water [Hard]

**Companies:** Amazon, Google, Meta, Microsoft

### Approach & Key Insight

**Problem: Given elevation map, compute how much water it can trap after rain.**

**Approach: Two Pointers (O(n) space O(1))**

**- Maintain leftMax and rightMax (max heights seen from each side)**

**- The water at any position = min(leftMax, rightMax) - height[pos]**

**- Move the pointer on the SMALLER max side: that side determines the water level**

**- If leftMax <= rightMax: water at left = leftMax - height[left]; advance left**

**- If rightMax < leftMax:  water at right = rightMax - height[right]; advance right**

**Alternative: Precompute leftMax[] and rightMax[] arrays — simpler to understand.**

**The two-pointer approach eliminates the need for those arrays.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(1) two-pointer / O(n) array approach |

### Java Solution

```
class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;

        while (left < right) {
            if (height[left] <= height[right]) {
                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    water += leftMax - height[left]; // trapped on left side
                }
                left++;
            } else {
                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    water += rightMax - height[right]; // trapped on right side
                }
                right--;
            }
        }
        return water;
    }

    // Alt: array approach (easier to reason about)
    public int trapArrays(int[] height) {
        int n = height.length;
        int[] lMax = new int[n], rMax = new int[n];
        lMax[0] = height[0];
        for (int i = 1; i < n; i++) lMax[i] = Math.max(lMax[i-1], height[i]);
        rMax[n-1] = height[n-1];
        for (int i = n-2; i >= 0; i--) rMax[i] = Math.max(rMax[i+1], height[i]);
        int water = 0;
```

## #7 Largest Rectangle in Histogram [Hard]

**Companies:** Amazon, Google

### Approach & Key Insight

**Problem: Find the largest rectangle area in a histogram.**

**Approach: Monotonic Stack**

**- Maintain a stack of indices in increasing height order**

**- When current bar is shorter than stack top: pop and calculate area**

**- Width = current index - stack.peek() - 1 (or current if stack empty)**

**- Height = height[popped index]**

**- After loop: empty stack (treat current as width boundary = n)**

**Key insight: For each bar as the shortest, the rectangle extends to the nearest**

**shorter bar on each side. Stack gives us those boundaries efficiently.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(n) |

### Java Solution

```
class Solution {

    public int largestRectangleArea(int[] heights) {

        Deque<Integer> stack = new ArrayDeque<>(); // stores indices

        int maxArea = 0;

        int n = heights.length;


        for (int i = 0; i <= n; i++) {

            int currH = (i == n) ? 0 : heights[i]; // sentinel 0 at end flushes stack


            while (!stack.isEmpty() && currH < heights[stack.peek()]) {

                int h = heights[stack.pop()];

                // Width: from stack top (exclusive) to i (exclusive)

                int w = stack.isEmpty() ? i : i - stack.peek() - 1;

                maxArea = Math.max(maxArea, h * w);

            }

            stack.push(i);

        }

        return maxArea;

    }

}
```

## #8 Sliding Window Maximum [Hard]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Find maximum in every window of size k in array.**

**Approach: Monotonic Deque (Decreasing)**

**- Maintain a deque of indices where values are in decreasing order**

**- Deque front = index of maximum in current window**

**- For each new element: remove from back all indices with smaller values (they're useless)**

**- Remove from front if out of window (index <= i - k)**

**- After processing first k elements, front is the window max**

**Key insight: Once a larger element enters the window, smaller elements to its left**

**can never be the maximum → safely discard them from the deque.**

| ⏱ **Time:** O(n) | 💾 **Space:** O(k) |
| --- | --- |

### Java Solution

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> dq = new ArrayDeque<>(); // stores INDICES, decreasing values

        for (int i = 0; i < n; i++) {
            // Remove indices outside window
            while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
            // Remove smaller elements from back (they can never be max)
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();

            dq.offerLast(i);
            // Start recording results after first full window
            if (i >= k - 1) result[i - k + 1] = nums[dq.peekFirst()];
        }
        return result;
    }
}
```

## #9 Product of Array Except Self [Medium]

**Companies:** Amazon, Microsoft, Apple, Google

### Approach & Key Insight

**Problem: Return array where each element is the product of all other elements. No division, O(n).**

**Approach: Prefix + Suffix Products (Two-Pass)**

**- Pass 1 (left to right): result[i] = product of all elements to the LEFT of i**

**- Pass 2 (right to left): multiply result[i] by product of all elements to the RIGHT of i**

**- This gives result[i] = leftProduct[i] * rightProduct[i] = product of all except i**

**Space optimization: use the output array for prefix products, single variable for suffix.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(1) excluding output |

### Java Solution

```java
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        // Pass 1: result[i] = product of all elements LEFT of i
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i - 1] * nums[i - 1];
        }

        // Pass 2: multiply by product of all elements RIGHT of i
        int rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= rightProduct;
            rightProduct *= nums[i];
        }
        return result;
    }
}
```

### #10 Top K Frequent Elements [Medium]

**Companies:** Amazon, Google, Meta, LinkedIn

#### Approach & Key Insight

**Problem: Return k most frequent elements. Better than O(n log n).**

**Approach 1: Bucket Sort (O(n))**

**- Count frequencies using HashMap**

**- Create buckets: bucket[freq] = list of numbers with that frequency**

**- Iterate buckets from highest freq down, collect k elements**

**Approach 2: Min-Heap (O(n log k))**

**- Count frequencies, maintain min-heap of size k**

**- Push all elements; pop when heap size > k**

**- More general; use when k is large or frequencies matter**

| | |
|---|---|
| ⏱ **Time:** O(n) bucket / O(n log k) heap | 💾 **Space:** O(n) |

### Java Solution

```
class Solution {
    // Approach 1: Bucket Sort — O(n)
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer,Integer> freq = new HashMap<>();
        for (int n : nums) freq.merge(n, 1, Integer::sum);

        // bucket[i] = list of numbers appearing exactly i times
        List<Integer>[] bucket = new List[nums.length + 1];
        for (int i = 0; i <= nums.length; i++) bucket[i] = new ArrayList<>();
        for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
            bucket[e.getValue()].add(e.getKey());
        }

        int[] result = new int[k];
        int idx = 0;
        for (int i = bucket.length - 1; i >= 0 && idx < k; i--) {
            for (int num : bucket[i]) {
                result[idx++] = num;
                if (idx == k) break;
            }
        }
        return result;
    }

    // Approach 2: Min-Heap — O(n log k)
    public int[] topKFrequentHeap(int[] nums, int k) {
        Map<Integer,Integer> freq = new HashMap<>();
        for (int n : nums) freq.merge(n, 1, Integer::sum);

        // Min-heap by frequency
        PriorityQueue<Integer> heap = new PriorityQueue<>(
            Comparator.comparingInt(freq::get));
        for (int num : freq.keySet()) {
            heap.offer(num);
```

# 🌳 Category II — Dynamic Programming

The most important interview pattern for senior-level interviews

## #11 Coin Change [Medium]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Find minimum number of coins to make amount. Return -1 if not possible.**

**Approach: Bottom-Up DP (Unbounded Knapsack)**

**- dp[i] = minimum coins needed to make amount i**

**- Initialize dp[0]=0, dp[1..amount]=∞ (impossible initially)**

**- For each amount i from 1 to amount:**

**- For each coin c: if c <= i, dp[i] = min(dp[i], dp[i-c] + 1)**

**- Return dp[amount] if reachable, else -1**

**Why unbounded: each coin can be used multiple times.**

**State transition: dp[i] = min(dp[i - coin] + 1) for all coins**

⏱ **Time:** O(amount × coins)

💾 **Space:** O(amount)

### Java Solution

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1); // use amount+1 as "infinity" proxy
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

## #12 Longest Increasing Subsequence [Medium]

**Companies:** Amazon, Google, Uber, Microsoft

### Approach & Key Insight

**Problem: Find length of longest strictly increasing subsequence.**

**Approach 1: DP — O(n²)**

**- dp[i] = length of LIS ending at index i**

**- dp[i] = max(dp[j] + 1) for all j < i where nums[j] < nums[i]**

**Approach 2: Patience Sorting (Binary Search) — O(n log n)**

**- Maintain tails[] array: tails[i] = smallest tail element of all IS of length i+1**

**- For each num: binary search for first tail >= num; replace it**

**- Length of tails = length of LIS**

**Why tails works: replacing maintains the invariant that tails is always sorted,**

**and the length of tails at any point equals the LIS length so far.**

| | |
|---|---|
| ⏱ **Time:** O(n log n) binary search | 💾 **Space:** O(n) |

### Java Solution

```
class Solution {
    // O(n log n) — Patience Sorting
    public int lengthOfLIS(int[] nums) {
        List<Integer> tails = new ArrayList<>();

        for (int num : nums) {
            // Binary search: find first tail >= num
            int lo = 0, hi = tails.size();
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                if (tails.get(mid) < num) lo = mid + 1;
                else hi = mid;
            }
            // lo is the position to place num
            if (lo == tails.size()) tails.add(num);   // extends LIS
            else tails.set(lo, num);                   // replaces with smaller tail
        }
        return tails.size();
    }

    // O(n²) DP — easier to understand
    public int lengthOfLIS_DP(int[] nums) {
        int n = nums.length, maxLen = 1;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            maxLen = Math.max(maxLen, dp[i]);
        }
```

## #13 Edit Distance [Hard]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Minimum operations (insert, delete, replace) to convert word1 to word2.**

**Approach: 2D Dynamic Programming**

**- dp[i][j] = min edits to convert word1[0..i-1] to word2[0..j-1]**

**- Base cases: dp[i][0]=i (delete all), dp[0][j]=j (insert all)**

**- If word1[i-1]==word2[j-1]: dp[i][j] = dp[i-1][j-1] (no operation needed)**

**- Else: dp[i][j] = 1 + min(dp[i-1][j], ← delete from word1**

**dp[i][j-1], ← insert into word1**

**dp[i-1][j-1]) ← replace in word1**

**Space optimization: only need previous row → O(n) space possible.**

| | |
|---|---|
| ⏱ **Time:** O(m×n) | 💾 **Space:** O(m×n) or O(n) optimized |

### Java Solution

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];

        // Base cases
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1]; // match — no operation
                } else {
                    dp[i][j] = 1 + Math.min(dp[i-1][j-1],    // replace
                                   Math.min(dp[i-1][j],       // delete
                                            dp[i][j-1]));     // insert
                }
            }
        }
        return dp[m][n];
    }
}
```

### #14 Regular Expression Matching [Hard]

**Companies:** Google, Meta, Amazon

#### Approach & Key Insight

**Problem: Implement regex matching with '.' (any char) and '*' (zero or more of preceding).**

**Approach: 2D DP**

- dp[i][j] = true if pattern p[0..j-1] matches string s[0..i-1]

- dp[0][0] = true (both empty)

- dp[0][j]: true if p[0..j-1] matches empty — only if pattern is like a*, a*b*, etc.

- For p[j-1] == '*': two choices:

  1. Zero occurrences of p[j-2]: dp[i][j] = dp[i][j-2]

  2. One+ occurrences: dp[i][j] = dp[i-1][j] if s[i-1] matches p[j-2]

- For normal char or '.': dp[i][j] = dp[i-1][j-1] if chars match

| | |
|---|---|
| ⏱ **Time:** O(m×n) | 💾 **Space:** O(m×n) |

### Java Solution

```
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;

        // Handle patterns like a*, a*b*, a*b*c* matching empty string
        for (int j = 2; j <= n; j++) {
            if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 2];
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                char sc = s.charAt(i - 1), pc = p.charAt(j - 1);
                if (pc == '*') {
                    // Zero occurrences of preceding element
                    dp[i][j] = dp[i][j - 2];
                    // One or more: if preceding char matches current s char
                    char prev = p.charAt(j - 2);
                    if (prev == '.' || prev == sc) {
                        dp[i][j] = dp[i][j] || dp[i - 1][j];
                    }
                } else if (pc == '.' || pc == sc) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }
        return dp[m][n];
    }
}
```

## #15 Word Break II [Hard]

**Companies:** Amazon, Google

### Approach & Key Insight

**Problem: Given string and dictionary, return all ways to break into valid words.**

**Approach: DFS + Memoization**

**- DFS from each position: try all words that match at current position**

**- Recursively solve remaining substring, prepend current word to each result**

**- Memoize: memo[start] = list of all valid sentences starting at index start**

**- This avoids recomputing same subproblem when multiple paths reach same index**

**Key: Without memoization this is O(2^n) worst case. With memo: O(n^2 × 2^n/2) typical.**

| | |
|---|---|
| ⏱ **Time:** O($n^2 \times 2^n$) worst case | 💾 **Space:** O(n × 2^n) memo |

### Java Solution

```
class Solution {
    public List<String> wordBreak(String s, List<String> wordDict) {
        Set<String> dict = new HashSet<>(wordDict);
        Map<Integer, List<String>> memo = new HashMap<>();
        return dfs(s, 0, dict, memo);
    }

    private List<String> dfs(String s, int start, Set<String> dict,
                             Map<Integer, List<String>> memo) {
        if (memo.containsKey(start)) return memo.get(start);

        List<String> result = new ArrayList<>();
        if (start == s.length()) {
            result.add(""); // base case: empty string
            return result;
        }

        for (int end = start + 1; end <= s.length(); end++) {
            String word = s.substring(start, end);
            if (dict.contains(word)) {
                List<String> rest = dfs(s, end, dict, memo);
                for (String r : rest) {
                    result.add(word + (r.isEmpty() ? "" : " " + r));
                }
            }
        }
        memo.put(start, result);
        return result;
    }
}
```

# 🔗 Category III — Graphs & Trees

BFS, DFS, Topological Sort, BST operations

## #16 Number of Islands [Medium]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Count number of islands (connected 1s) in a 2D binary grid.**

**Approach: DFS / BFS Flood Fill**

**- Iterate through each cell in grid**

**- When grid[r][c]=='1': increment island count, then DFS/BFS to mark all connected 1s as visited**

**- Mark visited cells as '0' (or use a visited[][] array)**

**- DFS explores all 4 directions from each cell**

**Union-Find is another valid approach — but DFS is most commonly expected.**

| | |
|---|---|
| ⏱ **Time:** O(m×n) | 💾 **Space:** O(m×n) recursion stack |

### Java Solution

```
class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[0].length; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c);
                }
            }
        }
        return count;
    }

    private void dfs(char[][] grid, int r, int c) {
        // Bounds check + water check
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length
            || grid[r][c] != '1') return;

        grid[r][c] = '0'; // mark visited
        dfs(grid, r+1, c);
        dfs(grid, r-1, c);
        dfs(grid, r, c+1);
        dfs(grid, r, c-1);
    }
}
```

### #17 Course Schedule (Cycle Detection) [Medium]

**Companies:** Amazon, Google, Meta

#### Approach & Key Insight

**Problem: Can you finish all courses given prerequisites? (Detect cycle in directed graph)**

**Approach: Topological Sort (Kahn's Algorithm — BFS)**

**- Build adjacency list and in-degree array**

**- Start with all nodes of in-degree 0 (no prerequisites)**

**- BFS: process each 0-in-degree node, reduce neighbors' in-degrees**

**- If neighbor's in-degree becomes 0, add to queue**

**- If all courses processed: no cycle. If not: cycle exists.**

**Alternative: DFS with 3-color state (WHITE=unvisited, GRAY=in-stack, BLACK=done)**

| | |
|---|---|
| ⏱ **Time:** O(V+E) | 💾 **Space:** O(V+E) |

### Java Solution

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        int[] inDegree = new int[numCourses];

        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        for (int[] pre : prerequisites) {
            adj.get(pre[1]).add(pre[0]); // pre[1] → pre[0]
            inDegree[pre[0]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) queue.offer(i);
        }

        int processed = 0;
        while (!queue.isEmpty()) {
            int course = queue.poll();
            processed++;
            for (int next : adj.get(course)) {
                if (--inDegree[next] == 0) queue.offer(next);
            }
        }
        return processed == numCourses; // true = no cycle
    }
}
```

### #18 Word Ladder [Hard]

**Companies:** Amazon, Google, LinkedIn

#### Approach & Key Insight

**Problem: Find shortest transformation sequence from beginWord to endWord, changing one letter at a time.**

**Approach: BFS (shortest path in unweighted graph)**

**- Each word is a node; edges connect words differing by exactly one letter**

**- BFS guarantees shortest path**

**- Key optimization: for each position, try all 26 letters (instead of checking all word pairs)**

**- Use a Set for O(1) word lookup; remove words once visited to prevent revisiting**

**Bidirectional BFS (optimization): Search from both ends simultaneously — reduces search space from O(b^d) to O(b^(d/2)).**

| | |
|---|---|
| ⏱ **Time:** O(M² × N) where M=word length, N=wordList size | 💾 **Space:** O(M² × N) |

## Java Solution

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord)) return 0;

        Queue<String> queue = new LinkedList<>();
        queue.offer(beginWord);
        int steps = 1;

        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                String word = queue.poll();
                char[] chars = word.toCharArray();

                for (int j = 0; j < chars.length; j++) {
                    char original = chars[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        if (c == original) continue;
                        chars[j] = c;
                        String next = new String(chars);
                        if (next.equals(endWord)) return steps + 1;
                        if (wordSet.contains(next)) {
                            queue.offer(next);
                            wordSet.remove(next); // avoid revisiting
                        }
                        chars[j] = original; // restore
                    }
                }
            }
            steps++;
        }
        return 0; // no path found
    }
```

## #19 Serialize and Deserialize Binary Tree [Hard]

**Companies:** Google, Amazon, Meta

### Approach & Key Insight

**Problem: Design an algorithm to serialize/deserialize a binary tree.**

**Approach: BFS Level-Order Encoding**

**- Serialize: BFS traversal, use "null" for null nodes, comma-separated**

**- Deserialize: rebuild level by level using a queue**

**- Each non-null node gets two children from the data stream**

**- Null markers preserve tree structure**

**Alternative: Preorder DFS with null markers (simpler recursion).**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(n) |

### Java Solution

```
public class Codec {

    // Serialize using BFS level-order

    public String serialize(TreeNode root) {

        if (root == null) return "[]";

        StringBuilder sb = new StringBuilder("[");

        Queue<TreeNode> queue = new LinkedList<>();

        queue.offer(root);


        while (!queue.isEmpty()) {

            TreeNode node = queue.poll();

            if (node == null) {

                sb.append("null,");

            } else {

                sb.append(node.val).append(',');

                queue.offer(node.left);

                queue.offer(node.right);

            }

        }

        sb.setLength(sb.length() - 1); // remove trailing comma

        sb.append("]");

        return sb.toString();

    }


    // Deserialize: rebuild tree from BFS order

    public TreeNode deserialize(String data) {

        String inner = data.substring(1, data.length() - 1);

        if (inner.isEmpty() || inner.equals("null")) return null;


        String[] vals = inner.split(",");

        TreeNode root = new TreeNode(Integer.parseInt(vals[0]));

        Queue<TreeNode> queue = new LinkedList<>();

        queue.offer(root);

        int i = 1;
```

## #20 Find Median from Data Stream [Hard]

**Companies:** Amazon, Google

### Approach & Key Insight

**Problem: Design data structure that supports addNum and findMedian in O(log n) and O(1).**

**Approach: Two Heaps (Max-Heap + Min-Heap)**

**- Max-Heap (lo): stores smaller half. Top = max of smaller half.**

**- Min-Heap (hi): stores larger half. Top = min of larger half.**

**- Invariant: lo.size() == hi.size() OR lo.size() == hi.size() + 1**

**- Balance: always add to lo first (through hi), then balance sizes**

**- Median: if sizes equal → avg of tops; else → lo top**

**Java's PriorityQueue is a min-heap by default. Use Collections.reverseOrder() for max-heap.**

| | |
|---|---|
| ⏱ **Time:** O(log n) addNum, O(1) findMedian | 💾 **Space:** O(n) |

### Java Solution

```
class MedianFinder {

    // Max-heap for lower half, min-heap for upper half

    private PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());

    private PriorityQueue<Integer> hi = new PriorityQueue<>();


    public void addNum(int num) {

        lo.offer(num);                    // add to max-heap first

        hi.offer(lo.poll());              // push max of lo to hi (ensures lo <= hi
values)

        if (lo.size() < hi.size()) {      // rebalance: lo should have >= hi size

            lo.offer(hi.poll());

        }

    }


    public double findMedian() {

        if (lo.size() == hi.size()) {

            return (lo.peek() + hi.peek()) / 2.0;

        }

        return lo.peek(); // lo always has extra element when odd total

    }

}
```

# ⚙️ Category IV — Linked Lists, Stack & Design Problems

LRU, Reverse K-Group, Monotonic Stack patterns

## #21 LRU Cache [Medium]

**Companies:** Amazon, Google, Microsoft, Meta

### Approach & Key Insight

**Problem: Design O(1) get and put for Least Recently Used cache.**

**Approach: HashMap + Doubly Linked List**

**- HashMap: key → node (O(1) lookup)**

**- DLL: maintains access order — head=most recent, tail=least recent**

**- get: access node → move to head; return value**

**- put: if exists, update and move to head; if new, add to head and remove tail if over capacity**

**- DLL with sentinel head/tail nodes eliminates edge case checks**

**Java LinkedHashMap with removeEldestEntry can implement LRU in 3 lines — but interviewers want the manual implementation to test understanding.**

| ⏱ **Time:** O(1) get and put | 💾 **Space:** O(capacity) |
|---|---|

### Java Solution

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0); // sentinel
    private final Node tail = new Node(0, 0); // sentinel

    static class Node {
        int key, val;
        Node prev, next;
        Node(int k, int v) { key = k; val = v; }
    }

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node n = map.get(key);
        moveToFront(n);
        return n.val;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            Node n = map.get(key);
            n.val = value;
            moveToFront(n);
        } else {
            Node n = new Node(key, value);
            map.put(key, n);
            addToFront(n);
```

## #22 Reverse Nodes in K-Group [Hard]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Reverse every k nodes of a linked list. Leave remainder as-is.**

**Approach: Iterative with dummy head**

**- Count k nodes ahead to check if full group exists**

**- If less than k remaining: leave as-is**

**- Reverse k nodes: standard reversal keeping track of prev, curr, next**

**- Connect reversed group back to previous group and advance pointers**

**- Key: save the tail of each reversed group (it becomes the connection point for next group)**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(1) |

### Java Solution

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prevGroupTail = dummy;

        while (true) {
            // Check if k nodes remain
            ListNode check = prevGroupTail;
            for (int i = 0; i < k; i++) {
                check = check.next;
                if (check == null) return dummy.next; // fewer than k nodes left
            }

            // Reverse k nodes
            ListNode groupHead = prevGroupTail.next;
            ListNode prev = null, curr = groupHead;
            for (int i = 0; i < k; i++) {
                ListNode next = curr.next;
                curr.next = prev;
                prev = curr;
                curr = next;
            }
            // Connect: prevGroupTail → new head (prev), groupHead → curr (next group)
            prevGroupTail.next = prev;
            groupHead.next = curr;
            prevGroupTail = groupHead; // groupHead is now the tail of reversed group
        }
    }
}
```

## #23 Merge K Sorted Lists [Hard]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Merge k sorted linked lists into one sorted list.**

**Approach: Min-Heap (Priority Queue)**

**- Add head of each list to a min-heap (sorted by node value)**

**- Extract minimum node, add to result, then push that node's next into heap**

**- Heap always contains the k current "candidates" (one per list)**

**- Repeat until heap is empty**

**Alternative: Divide and Conquer — merge pairs of lists repeatedly. O(n log k) same but more code.**

**The heap approach is more natural and commonly expected.**

| | |
|---|---|
| ⏱ **Time:** O(n log k) where n=total nodes, k=lists | 💾 **Space:** O(k) heap |

### Java Solution

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        // Min-heap comparing node values
        PriorityQueue<ListNode> heap = new PriorityQueue<>(
            Comparator.comparingInt(n -> n.val));

        // Add head of each non-empty list
        for (ListNode list : lists) {
            if (list != null) heap.offer(list);
        }

        ListNode dummy = new ListNode(0), curr = dummy;
        while (!heap.isEmpty()) {
            ListNode node = heap.poll();
            curr.next = node;
            curr = curr.next;
            if (node.next != null) heap.offer(node.next); // push next candidate
        }
        return dummy.next;
    }
}
```

# 🔄 Category V — Backtracking, Binary Search & Heap

N-Queens, Subsets, Median, Job Scheduling

## #24 N-Queens [Hard]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Place n queens on n×n board with no two attacking each other. Return all solutions.**

**Approach: Backtracking with Set-based Conflict Checking**

**- Place queens row by row (one per row — already handles row conflict)**

**- Track: cols (used columns), diagonals (row-col), anti-diagonals (row+col)**

**- Same diagonal: row-col is constant. Same anti-diagonal: row+col is constant.**

**- Try each column for current row; skip if column/diagonal/anti-diagonal is used**

**- When all rows filled: valid solution found**

**Pruning: Three sets give O(1) conflict check vs O(n) array scan.**

| ⏱ **Time:** O(n!) solutions explored | 💾 **Space:** O(n) stack + sets |
|---|---|

### Java Solution

```
class Solution {

    public List<List<String>> solveNQueens(int n) {

        List<List<String>> result = new ArrayList<>();

        Set<Integer> cols = new HashSet<>(), diag = new HashSet<>(), antiDiag = new
HashSet<>();

        int[] queens = new int[n]; // queens[row] = col

        backtrack(0, n, queens, cols, diag, antiDiag, result);

        return result;

    }


    private void backtrack(int row, int n, int[] queens,

                           Set<Integer> cols, Set<Integer> diag, Set<Integer> antiDiag,

                           List<List<String>> result) {

        if (row == n) {

            result.add(buildBoard(queens, n));

            return;

        }

        for (int col = 0; col < n; col++) {

            if (cols.contains(col) || diag.contains(row-col) ||
antiDiag.contains(row+col))

                continue;


            cols.add(col); diag.add(row-col); antiDiag.add(row+col);

            queens[row] = col;

            backtrack(row+1, n, queens, cols, diag, antiDiag, result);

            cols.remove(col); diag.remove(row-col); antiDiag.remove(row+col);

        }

    }


    private List<String> buildBoard(int[] queens, int n) {

        List<String> board = new ArrayList<>();

        for (int r = 0; r < n; r++) {

            char[] row = new char[n];

            Arrays.fill(row, '.');

            row[queens[r]] = 'Q';
```

## #25 Median of Two Sorted Arrays [Hard]

**Companies:** Google, Amazon, Microsoft

### Approach & Key Insight

**Problem: Find median of two sorted arrays in O(log(m+n)) time.**

**Approach: Binary Search on the smaller array**

**- Partition both arrays so that left halves have (m+n)/2 elements total**

**- Binary search partition of smaller array; derive other array's partition**

**- Valid partition: maxLeft1 <= minRight2 AND maxLeft2 <= minRight1**

**- If too many from array1 on left: move partition left (hi = partX - 1)**

**- If too few: move right (lo = partX + 1)**

**- Median from the boundary elements of the two partitions**

**This is the hardest pure algorithm in FAANG interviews. Understanding is more important than memorizing.**

| | |
|---|---|
| ⏱ **Time:** O(log(min(m,n))) | 💾 **Space:** O(1) |

### Java Solution

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        // Ensure nums1 is the smaller array
        if (nums1.length > nums2.length)
            return findMedianSortedArrays(nums2, nums1);

        int m = nums1.length, n = nums2.length;
        int lo = 0, hi = m;
        int half = (m + n + 1) / 2; // left half size (ceiling)

        while (lo <= hi) {
            int partX = lo + (hi - lo) / 2;  // partition of nums1
            int partY = half - partX;          // partition of nums2

            int maxLeft1  = (partX == 0) ? Integer.MIN_VALUE : nums1[partX - 1];
            int minRight1 = (partX == m) ? Integer.MAX_VALUE : nums1[partX];
            int maxLeft2  = (partY == 0) ? Integer.MIN_VALUE : nums2[partY - 1];
            int minRight2 = (partY == n) ? Integer.MAX_VALUE : nums2[partY];

            if (maxLeft1 <= minRight2 && maxLeft2 <= minRight1) {
                // Found correct partition
                int maxLeft = Math.max(maxLeft1, maxLeft2);
                if ((m + n) % 2 == 1) return maxLeft; // odd total
                int minRight = Math.min(minRight1, minRight2);
                return (maxLeft + minRight) / 2.0;
            } else if (maxLeft1 > minRight2) {
                hi = partX - 1; // too many from nums1 on left
            } else {
                lo = partX + 1; // too few from nums1 on left
            }
        }
        return 0.0;
    }
}
```

## #26 Meeting Rooms II [Medium]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Minimum number of conference rooms required for given meeting intervals.**

**Approach: Min-Heap (Sort + Greedy)**

**- Sort meetings by start time**

**- Use min-heap to track end times of ongoing meetings**

**- For each meeting: if heap top <= current start, room is free (pop and reuse)**

**- Otherwise: need a new room (push)**

**- Heap size at any point = rooms currently in use; max size = answer**

**Alternative: Sweep Line — count events (start=+1, end=-1), track running sum.**

| ⏱ **Time:** O(n log n) | 💾 **Space:** O(n) |
| --- | --- |

### Java Solution

```
class Solution {

    public int minMeetingRooms(int[][] intervals) {

        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

        // Min-heap of end times

        PriorityQueue<Integer> heap = new PriorityQueue<>();


        for (int[] interval : intervals) {

            // Free a room if earliest-ending meeting ends before this one starts

            if (!heap.isEmpty() && heap.peek() <= interval[0]) {

                heap.poll();

            }

            heap.offer(interval[1]); // occupy room, track end time

        }

        return heap.size(); // rooms still in use = total rooms needed

    }


    // Alternative: Sweep Line

    public int minMeetingRoomsSweep(int[][] intervals) {

        int n = intervals.length;

        int[] starts = new int[n], ends = new int[n];

        for (int i = 0; i < n; i++) { starts[i] = intervals[i][0]; ends[i] =
intervals[i][1]; }

        Arrays.sort(starts); Arrays.sort(ends);


        int rooms = 0, maxRooms = 0, endPtr = 0;

        for (int start : starts) {

            if (start < ends[endPtr]) rooms++;       // new meeting starts before
earliest end

            else { rooms = rooms; endPtr++; }         // reuse a room

            maxRooms = Math.max(maxRooms, rooms);

        }

        return maxRooms;

    }

}
```

## #27 Task Scheduler [Medium]

**Companies:** Amazon, Google

### Approach & Key Insight

**Problem: Schedule tasks (each takes 1 unit, cooldown n between same task). Find minimum time.**

**Approach: Greedy — Count Most Frequent Task**

**- Most frequent task determines the structure: (maxFreq - 1) * (n + 1) + countOfMaxFreq**

**- Formula gives minimum time IF we have enough tasks to fill all idle slots**

**- If we have many diverse tasks: answer is simply total number of tasks (no idle needed)**

**- max(formula_result, total_tasks)**

**Why: Build a schedule with (maxFreq-1) "chunks" of (n+1) slots each, plus a final group.**

**If other tasks fill the idle slots completely, the extra tasks just extend the time.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(1) only 26 chars |

### Java Solution

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] freq = new int[26];
        for (char t : tasks) freq[t - 'A']++;
        Arrays.sort(freq);

        int maxFreq = freq[25];
        // Count how many tasks share the max frequency
        int countMax = 0;
        for (int f : freq) if (f == maxFreq) countMax++;

        // Slots needed = (maxFreq-1) full chunks + last partial chunk
        int minTime = (maxFreq - 1) * (n + 1) + countMax;
        // If tasks are dense: no idle needed, answer is just task count
        return Math.max(minTime, tasks.length);
    }
}
```

## #28 Maximum Profit in Job Scheduling [Hard]

**Companies:** Google, Amazon

### Approach & Key Insight

**Problem: Given jobs with start, end, profit, find max profit from non-overlapping jobs.**

**Approach: DP + Binary Search**

**- Sort jobs by end time**

**- dp[i] = max profit considering first i jobs**

**- For each job i: two choices:**

**1. Skip job i: dp[i] = dp[i-1]**

**2. Take job i: dp[i] = profit[i] + dp[lastNonConflict(i)]**

- lastNonConflict: latest job that ends <= start[i] (binary search on end times)

- Take the max of both choices

This is interval scheduling maximization — a key pattern for scheduling problems.

| ⏱ **Time:** O(n log n) | 💾 **Space:** O(n) |
| --- | --- |

## Java Solution

```
class Solution {

    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {

        int n = startTime.length;

        int[][] jobs = new int[n][3];

        for (int i = 0; i < n; i++) jobs[i] = new int[]{endTime[i], startTime[i],
profit[i]};

        Arrays.sort(jobs, Comparator.comparingInt(j -> j[0])); // sort by end time


        int[] dp = new int[n + 1]; // dp[i] = max profit with first i jobs


        for (int i = 1; i <= n; i++) {

            int end = jobs[i-1][0], start = jobs[i-1][1], prof = jobs[i-1][2];

            // Binary search: latest job ending <= start of current job

            int lo = 0, hi = i - 1;

            while (lo < hi) {

                int mid = lo + (hi - lo + 1) / 2;

                if (jobs[mid-1][0] <= start) lo = mid;

                else hi = mid - 1;

            }

            // lo = count of compatible previous jobs

            dp[i] = Math.max(dp[i-1],        // skip this job

                             prof + dp[lo]);   // take this job

        }

        return dp[n];

    }

}
```

# 🔤 Category VI — String DP, Stack & Miscellaneous Hard Problems

Palindrome Partition, Basic Calculator, Alien Dictionary

## #29 Alien Dictionary [Hard]

**Companies:** Google, Amazon, Meta

### Approach & Key Insight

**Problem: Given sorted alien language words, find the character ordering of the alien alphabet.**

**Approach: Topological Sort**

**- Compare adjacent words to extract ordering constraints (a before b means a→b edge)**

**- Build directed graph; detect cycle (invalid input) using DFS or in-degree tracking**

**- Topological sort gives the character order**

**Key edge cases:**

**- "abc" before "ab" is invalid (longer word first with same prefix)**

**- Multiple valid orderings exist — return any**

⏱ **Time:** O(C) where C = total chars in all words

💾 **Space:** O(1) only 26 chars

### Java Solution

```
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> adj = new HashMap<>();
        Map<Character, Integer> inDegree = new HashMap<>();

        // Initialize all characters
        for (String w : words) for (char c : w.toCharArray()) {
            adj.putIfAbsent(c, new HashSet<>());
            inDegree.putIfAbsent(c, 0);
        }

        // Extract ordering from adjacent word pairs
        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i+1];
            int len = Math.min(w1.length(), w2.length());
            if (w1.length() > w2.length() && w1.startsWith(w2)) return ""; // invalid
            for (int j = 0; j < len; j++) {
                if (w1.charAt(j) != w2.charAt(j)) {
                    char from = w1.charAt(j), to = w2.charAt(j);
                    if (!adj.get(from).contains(to)) {
                        adj.get(from).add(to);
                        inDegree.merge(to, 1, Integer::sum);
                    }
                    break;
                }
            }
        }

        // Topological sort (Kahn's BFS)
        Queue<Character> queue = new LinkedList<>();
        for (char c : inDegree.keySet()) if (inDegree.get(c) == 0) queue.offer(c);

        StringBuilder result = new StringBuilder();
        while (!queue.isEmpty()) {
```

## #30 Longest Palindromic Substring [Medium]

**Companies:** Amazon, Microsoft, Apple

### Approach & Key Insight

**Problem: Find longest palindromic substring in s.**

**Approach: Expand Around Center**

**- For each possible center (n centers for odd, n-1 for even length palindromes):**

**- Expand outward while s[left]==s[right]**

**- Track longest palindrome found**

**- 2n-1 possible centers total (each char + each gap between chars)**

**Alternative: Manacher's Algorithm — O(n) but complex to implement.**

**Expand Around Center is $O(n^2)$ time and simpler — preferred in interviews.**

| | |
|---|---|
| ⏱ **Time:** $O(n^2)$ | 💾 **Space:** O(1) |

### Java Solution

```
class Solution {
    private int start = 0, maxLen = 1;

    public String longestPalindrome(String s) {
        if (s.length() < 2) return s;
        for (int i = 0; i < s.length(); i++) {
            expand(s, i, i);     // odd length palindromes
            expand(s, i, i + 1); // even length palindromes
        }
        return s.substring(start, start + maxLen);
    }

    private void expand(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            if (right - left + 1 > maxLen) {
                start = left;
                maxLen = right - left + 1;
            }
            left--; right++;
        }
    }
}
```

# Quick Reference — Complexity Cheatsheet & Patterns

| Pattern | When to Use | Time | Space |
|---|---|---|---|
| HashMap (Two Sum style) | Find complement, count freq, grouping | O(n) | O(n) |
| Two Pointers | Sorted array, pairs, triplets, containers | O(n) or $O(n^2)$ | O(1) |
| Sliding Window | Subarray/substring with constraint | O(n) | O(1) or O(k) |
| Monotonic Stack | Next greater/smaller element, histogram | O(n) | O(n) |
| Monotonic Deque | Sliding window max/min | O(n) | O(k) |
| BFS (Level Order) | Shortest path, level traversal, word ladder | O(V+E) | O(V) |
| DFS + Backtracking | Combinations, permutations, N-Queens | O(n!) | O(n) stack |
| Binary Search | Sorted array, rotated array, answer space | O(log n) | O(1) |
| Topological Sort | Course schedule, alien dict, build order | O(V+E) | O(V+E) |
| Prefix Sum | Subarray sum, range queries | O(n) preprocess | O(n) |
| DP (1D) | Coin change, LIS, decode ways | O(n) or $O(n^2)$ | O(n) |
| DP (2D) | Edit distance, regex, unique paths | O(m×n) | O(m×n) |
| Min/Max Heap | Top K, merge K lists, median | O(n log k) | O(k) |
| Two Heaps | Running median, data stream | O(log n) | O(n) |
| Trie | Word search, autocomplete, prefix matching | O(m) per op | O(ALPHABET×n) |
| Union-Find | Connected components, cycle detection | O(α(n)) ≈ O(1) | O(n) |

## Interview Strategy Tips

| Tip | Details |
|---|---|
| Clarify before coding | Ask about constraints: size, duplicates, negative numbers, empty input |
| Start brute force | Describe $O(n^2)$ or O(2^n) solution first, then optimize |
| Think out loud | Interviewers care about your thought process, not just the answer |
| Test with examples | Walk through your code with 2-3 examples before declaring done |
| Handle edge cases | Empty input, single element, all same elements, sorted/reverse sorted |
| Know complexities | Always state and justify time AND space complexity |
| Pattern recognition | Sorted+pairs → Two Pointers; substring → Sliding Window; k-th → Heap |

| Tip | Details |
|---|---|
| DP state definition | Define dp[i] precisely before writing transitions |
| Graph problems | Always ask: directed/undirected? weighted? cycles possible? |
| Linked list | Use dummy head node to simplify edge cases at list boundaries |

**50 Most Frequently Asked LeetCode Medium & Hard Problems — FAANG + Top Tech Companies | 2025 Edition**

**Amazon · Google · Meta · Microsoft · Apple · Netflix · Uber · LinkedIn · Adobe | Java Solutions**

# 🔍 Category VII — Binary Search

Classic binary search problems that appear in every big-tech interview

## #31 Find Minimum in Rotated Sorted Array [Medium]

**Companies:** Microsoft, Amazon, Meta

### Approach & Key Insight

**Problem: Find minimum element in a rotated sorted array with no duplicates.**

**Approach: Binary Search**

**- If array not rotated (nums[lo] < nums[hi]): minimum is nums[lo]**

**- Otherwise: minimum is in the rotated segment**

**- Compare mid with hi: if nums[mid] > nums[hi], min is in right half**

**Otherwise min is in left half (including mid — mid could be the min)**

**- Invariant: minimum is always within [lo, hi]**

**Key: compare mid with hi (not lo) to correctly identify which side is sorted.**

| ⏱ Time: O(log n) | 💾 Space: O(1) |
|---|---|

### Java Solution

```java
class Solution {
    public int findMin(int[] nums) {
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] > nums[hi]) {
                lo = mid + 1;  // min is in right half
            } else {
                hi = mid;      // min is mid or in left half
            }
        }
        return nums[lo]; // lo == hi == index of minimum
    }
}
```

### #32 Search in Rotated Sorted Array [Medium]

**Companies:** Amazon, Google, Meta

#### Approach & Key Insight

**Problem: Search for target in rotated sorted array. Return index or -1.**

**Approach: Binary Search — identify which half is sorted**

**- One of the two halves is ALWAYS sorted after splitting at mid**

**- If nums[lo] <= nums[mid]: left half is sorted**

**- If target in [nums[lo], nums[mid]]: search left → hi = mid - 1**

**- Else: search right → lo = mid + 1**

**- Else: right half is sorted**

**- If target in [nums[mid+1], nums[hi]]: search right → lo = mid + 1**

**- Else: search left → hi = mid - 1**

**This single binary search handles all rotation cases.**

| ⏱ **Time:** O(log n) | 💾 **Space:** O(1) |
| --- | --- |

### Java Solution

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;

            if (nums[lo] <= nums[mid]) {            // left half sorted
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {                                 // right half sorted
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }
}
```

# 📐 Category VIII — Greedy & Interval Problems

Merge Intervals, Jump Game, Schedule — key greedy patterns

## #33 Merge Intervals [Medium]

**Companies:** Google, Facebook, Amazon

### Approach & Key Insight

**Problem: Merge all overlapping intervals. Return merged array.**

**Approach: Sort + Linear Sweep**

**- Sort intervals by start time**

**- Maintain current merged interval; for each next interval:**

**- If next.start <= current.end: overlapping → extend current.end = max(current.end, next.end)**

**- Else: no overlap → add current to result, start new current with next interval**

**- Add the last current interval to result**

**Key: sort first! Without sorting, checking overlaps requires O(n²) comparisons.**

| ⏱ **Time:** O(n log n) | 💾 **Space:** O(n) output |
|---|---|

### Java Solution

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
        List<int[]> merged = new ArrayList<>();

        int[] curr = intervals[0];
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] <= curr[1]) {
                // Overlapping: extend end
                curr[1] = Math.max(curr[1], intervals[i][1]);
            } else {
                // Non-overlapping: save current, move to next
                merged.add(curr);
                curr = intervals[i];
            }
        }
        merged.add(curr);
        return merged.toArray(new int[0][]);
    }
}
```

### #34 Jump Game II [Medium]

**Companies:** Amazon, Google

#### Approach & Key Insight

**Problem: Minimum jumps to reach last index. Array values = max jump from that index.**

**Approach: Greedy BFS (treat as implicit BFS levels)**

**- Think of it as BFS layers: current layer = positions reachable in 'jumps' steps**

**- Track: jumps, currentEnd (furthest reachable in current jump), farthest (overall max reach)**

**- When i reaches currentEnd: increment jumps, update currentEnd = farthest**

**- Each "level" = one jump; we want minimum levels to reach end**

**This greedy works because we always extend to the farthest reachable position.**

| ⏱ **Time:** O(n) | 💾 **Space:** O(1) |
| --- | --- |

### Java Solution

```
class Solution {
    public int jump(int[] nums) {
        int jumps = 0, currentEnd = 0, farthest = 0;
        // Don't need to jump from last index
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == currentEnd) {         // exhausted current jump's range
                jumps++;
                currentEnd = farthest;     // jump as far as possible
                if (currentEnd >= nums.length - 1) break; // reached end
            }
        }
        return jumps;
    }
}
```

# 🌐 Category IX — Advanced Graph Problems

Pacific Atlantic, Clone Graph, Alien Dictionary variants

## #35 Pacific Atlantic Water Flow [Medium]

**Companies:** Google, Amazon

### Approach & Key Insight

**Problem: Find cells from which water can flow to BOTH Pacific and Atlantic oceans.**

**Approach: Reverse BFS/DFS from both oceans simultaneously**

**- Instead of simulating water flow downhill, reverse: flow UPHILL from oceans**

**- Start BFS/DFS from Pacific border cells, mark all reachable-from-Pacific cells**

**- Start BFS/DFS from Atlantic border cells, mark all reachable-from-Atlantic cells**

**- Result: cells reachable from both**

**Key insight: "can reach ocean" (downhill) is equivalent to "ocean can reach cell" (uphill ≥).**

**Reversing the direction avoids repeatedly searching from every cell.**

| ⏱ **Time:** O(m×n) | 💾 **Space:** O(m×n) |
|---|---|

### Java Solution

```
class Solution {
    private int[][] DIRS = {{0,1},{0,-1},{1,0},{-1,0}};

    public List<List<Integer>> pacificAtlantic(int[][] heights) {
        int m = heights.length, n = heights[0].length;
        boolean[][] pacific = new boolean[m][n];
        boolean[][] atlantic = new boolean[m][n];

        Queue<int[]> pQueue = new LinkedList<>(), aQueue = new LinkedList<>();
        // Pacific: top row + left col; Atlantic: bottom row + right col
        for (int i = 0; i < m; i++) {
            pQueue.offer(new int[]{i, 0});     pacific[i][0] = true;
            aQueue.offer(new int[]{i, n-1});   atlantic[i][n-1] = true;
        }
        for (int j = 0; j < n; j++) {
            pQueue.offer(new int[]{0, j});     pacific[0][j] = true;
            aQueue.offer(new int[]{m-1, j});   atlantic[m-1][j] = true;
        }
        bfs(heights, pQueue, pacific);
        bfs(heights, aQueue, atlantic);

        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (pacific[i][j] && atlantic[i][j])
                    result.add(Arrays.asList(i, j));
        return result;
    }

    private void bfs(int[][] h, Queue<int[]> q, boolean[][] visited) {
        while (!q.isEmpty()) {
            int[] cell = q.poll();
            for (int[] d : DIRS) {
                int r = cell[0]+d[0], c = cell[1]+d[1];
```

## #36 Clone Graph [Medium]

**Companies:** Meta, Amazon, Google

### Approach & Key Insight

**Problem: Deep clone a connected undirected graph. Each node has val and list of neighbors.**

**Approach: BFS with HashMap (original → clone)**

**- HashMap maps each original node to its cloned counterpart**

**- BFS traversal: for each node, ensure its clone exists**

**- For each neighbor of original: ensure neighbor's clone exists; add to clone's neighbors**

**- HashMap also acts as visited set — prevents infinite loops in cyclic graphs**

| ⏱ **Time:** O(V+E) | 💾 **Space:** O(V) |
| --- | --- |

### Java Solution

```
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        Map<Node, Node> cloneMap = new HashMap<>(); // original → clone

        Queue<Node> queue = new LinkedList<>();
        queue.offer(node);
        cloneMap.put(node, new Node(node.val));

        while (!queue.isEmpty()) {
            Node curr = queue.poll();
            for (Node neighbor : curr.neighbors) {
                if (!cloneMap.containsKey(neighbor)) {
                    cloneMap.put(neighbor, new Node(neighbor.val));
                    queue.offer(neighbor); // visit neighbor
                }
                // Connect clone's neighbor
                cloneMap.get(curr).neighbors.add(cloneMap.get(neighbor));
            }
        }
        return cloneMap.get(node);
    }
}
```

# 🌲 Category X — Tree Problems

BST, Construct from Traversals, Validate BST

## #37 Validate Binary Search Tree [Medium]

**Companies:** Amazon, Microsoft, Google

### Approach & Key Insight

**Problem: Determine if a binary tree is a valid BST.**

**Approach: DFS with Min/Max Bounds**

**- Each node must be within (min, max) bounds inherited from parent**

**- Root: (-∞, +∞). Left child: (min, node.val). Right child: (node.val, max)**

**- A node is valid if: min < node.val < max**

**- Recursively validate with updated bounds**

**Common mistake: just checking left.val < root.val < right.val — fails for the case where**

**a right subtree has a node smaller than the root's ancestor.**

| ⏱ **Time:** O(n) | 💾 **Space:** O(n) stack |
|---|---|

### Java Solution

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;
        return validate(node.left, min, node.val) &&
               validate(node.right, node.val, max);
    }
}
```

## #38 Construct Binary Tree from Preorder and Inorder Traversal [Medium]

**Companies:** Amazon, Microsoft

### Approach & Key Insight

**Problem: Build binary tree from preorder and inorder traversal arrays.**

**Approach: Recursive with HashMap**

**- Preorder: first element is always the ROOT**

**- Inorder: root divides array into left subtree (left of root) and right subtree (right)**

**- Use HashMap for O(1) index lookup in inorder array**

**- Recurse: left subtree uses preorder[1..leftSize] and inorder[0..rootIdx-1]**

**right subtree uses rest of preorder and inorder[rootIdx+1..]**

**- Track preorder index with a pointer (or pass explicit boundaries)**

⏱ **Time:** O(n)

💾 **Space:** O(n) map + stack

### Java Solution

```
class Solution {
    private Map<Integer, Integer> inorderMap = new HashMap<>();
    private int preIdx = 0;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i < inorder.length; i++) inorderMap.put(inorder[i], i);
        return build(preorder, 0, inorder.length - 1);
    }

    private TreeNode build(int[] preorder, int inLeft, int inRight) {
        if (inLeft > inRight) return null;
        int rootVal = preorder[preIdx++];
        TreeNode root = new TreeNode(rootVal);
        int mid = inorderMap.get(rootVal); // split point in inorder
        root.left  = build(preorder, inLeft, mid - 1);   // build left subtree
        root.right = build(preorder, mid + 1, inRight);  // build right subtree
        return root;
    }
}
```

## #39 Kth Smallest Element in BST [Medium]

**Companies:** Google, Amazon, LinkedIn

### Approach & Key Insight

**Problem: Find kth smallest element in a BST.**

**Approach: Inorder Traversal (gives sorted order in BST)**

**- Inorder traversal of BST = ascending sorted sequence**

**- Return the kth element visited during inorder traversal**

**- Can do iterative (stack-based) to stop early at kth element**

**without traversing the entire tree**

**Follow-up: If BST is modified frequently and findKth called often → augment each node with a count of left subtree nodes → O(log n) per query.**

| | |
|---|---|
| ⏱ **Time:** O(H+k) where H=height | 💾 **Space:** O(H) stack |

### Java Solution

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        // Iterative inorder — stop early at kth
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;

        while (curr != null || !stack.isEmpty()) {
            // Go as far left as possible
            while (curr != null) { stack.push(curr); curr = curr.left; }
            curr = stack.pop();
            if (--k == 0) return curr.val; // kth smallest found
            curr = curr.right;
        }
        return -1;
    }
}
```

# Category XI — String & Stack Problems

Word Search, Group Anagrams, Basic Calculator

## #40 Word Search [Medium]

**Companies:** Amazon, Microsoft, Google

### Approach & Key Insight

**Problem: Given a 2D board of characters, find if word exists as adjacent cells path.**

**Approach: DFS + Backtracking**

**- For each cell, try starting DFS if it matches word[0]**

**- DFS: check current cell matches word[idx]; mark visited (in-place with '#')**

**- Recursively try 4 directions for next character**

**- Restore cell after backtracking (replace '#' with original char)**

**- Return true if all characters matched in sequence**

**Optimization: check first and last char frequency before DFS — prune impossible cases.**

| | |
|---|---|
| ⏱ **Time:** O(m×n×4^L) where L=word length | 💾 **Space:** O(L) recursion |

### Java Solution

```
class Solution {
    private char[][] board;
    private String word;

    public boolean exist(char[][] board, String word) {
        this.board = board;
        this.word = word;
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                if (dfs(r, c, 0)) return true;
        return false;
    }

    private boolean dfs(int r, int c, int idx) {
        if (idx == word.length()) return true;
        if (r<0||r>=board.length||c<0||c>=board[0].length) return false;
        if (board[r][c] != word.charAt(idx)) return false;

        char temp = board[r][c];
        board[r][c] = '#';  // mark visited in-place

        boolean found = dfs(r+1,c,idx+1) || dfs(r-1,c,idx+1)
                      || dfs(r,c+1,idx+1) || dfs(r,c-1,idx+1);

        board[r][c] = temp;  // restore (backtrack)
        return found;
    }
}
```

### #41 Group Anagrams [Medium]

**Companies:** Amazon, Google, Microsoft

#### Approach & Key Insight

**Problem: Group strings that are anagrams of each other.**

**Approach: HashMap with sorted string as key**

**- Sort each string → anagrams produce same sorted key**

**- Map sorted_key → list of original strings**

**- Return all map values**

**Alternative key: character frequency array as key (avoids O(k log k) sort).**

**Use "2#0#1#..." style string key from 26-char frequency count → O(n×k) total.**

| | |
|---|---|
| ⏱ **Time:** O(n×k log k) sort / O(n×k) freq | 💾 **Space:** O(n×k) |

### Java Solution

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }

    // O(n*k) approach using character frequency as key
    public List<List<String>> groupAnagramsFreq(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            int[] count = new int[26];
            for (char c : s.toCharArray()) count[c - 'a']++;
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < 26; i++) sb.append(count[i]).append('#');
            String key = sb.toString();
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

### #42 Basic Calculator II [Medium]

**Companies:** Amazon, Google

#### Approach & Key Insight

**Problem: Evaluate string expression with +, -, *, / and spaces. No parentheses.**

**Approach: Stack**

**- Process digits to build current number**

**- On operator or end: apply PREVIOUS operator to current number**

**  * + : push number onto stack**

**  * - : push negated number onto stack**

**  * * : pop top, multiply with current, push back**

**  * / : pop top, divide by current (truncate toward zero), push back**

**- Final answer: sum all values in stack**

**- Multiplication/division have higher precedence — handle immediately via stack pop.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(n) |

## Java Solution

```java
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int num = 0;
        char op = '+'; // start with implicit '+'

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (Character.isDigit(c)) {
                num = num * 10 + (c - '0');
            }

            if ((!Character.isDigit(c) && c != ' ') || i == s.length() - 1) {
                switch (op) {
                    case '+': stack.push(num);  break;
                    case '-': stack.push(-num); break;
                    case '*': stack.push(stack.pop() * num); break;
                    case '/': stack.push(stack.pop() / num); break;
                }
                op = c;   // update operator for next number
                num = 0;  // reset current number
            }
        }

        int result = 0;
        while (!stack.isEmpty()) result += stack.pop();
        return result;
    }
}
```

# 🎒 Category XII — Backtracking & Combinations

Subsets II, Decode Ways, Palindrome Partition

## #43 Subsets II [Medium]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Return all subsets of array that may contain duplicates. No duplicate subsets.**

**Approach: Backtracking with Deduplication**

**- Sort array first (groups duplicates together)**

**- At each level: skip duplicate values at the SAME recursion level**

**(but allow same value at DIFFERENT levels — it's a new subset)**

**- Dedup condition: if i > start && nums[i] == nums[i-1]: skip**

**'i > start' means we're not at the first element of this recursion level**

**Pattern: Sort + skip duplicates at same level is the universal dedup pattern for backtracking.**

| | |
|---|---|
| ⏱ **Time:** O(2^n) | 💾 **Space:** O(n) stack |

### Java Solution

```
class Solution {

    public List<List<Integer>> subsetsWithDup(int[] nums) {

        Arrays.sort(nums); // MUST sort to group duplicates

        List<List<Integer>> result = new ArrayList<>();

        backtrack(nums, 0, new ArrayList<>(), result);

        return result;

    }


    private void backtrack(int[] nums, int start, List<Integer> curr,
List<List<Integer>> result) {

        result.add(new ArrayList<>(curr));


        for (int i = start; i < nums.length; i++) {

            // Skip duplicate at same level (not same depth)

            if (i > start && nums[i] == nums[i - 1]) continue;

            curr.add(nums[i]);

            backtrack(nums, i + 1, curr, result);

            curr.remove(curr.size() - 1); // backtrack

        }

    }

}
```

## #44 Decode Ways [Medium]

**Companies:** Meta, Amazon, Google

### Approach & Key Insight

**Problem: Count ways to decode a digit string into letters (1→A, 26→Z).**

**Approach: Dynamic Programming (1D)**

**- dp[i] = number of ways to decode s[0..i-1]**

**- dp[0] = 1 (empty string has one decoding)**

**- dp[1] = 1 if s[0]!='0', else 0**

**- For i >= 2:**

- One-digit decode: if s[i-1]!='0', dp[i] += dp[i-1]

- Two-digit decode: if s[i-2..i-1] in [10,26], dp[i] += dp[i-2]

- Edge case: '0' can only be decoded as part of '10' or '20'

| ⏱ **Time:** O(n) | 💾 **Space:** O(n) or O(1) with 2 variables |
|---|---|

### Java Solution

```
class Solution {
    public int numDecodings(String s) {
        if (s.isEmpty() || s.charAt(0) == '0') return 0;
        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1;  // empty string
        dp[1] = 1;  // first char (already checked not '0')

        for (int i = 2; i <= n; i++) {
            int oneDigit = s.charAt(i-1) - '0';
            int twoDigit = Integer.parseInt(s.substring(i-2, i));

            if (oneDigit >= 1) dp[i] += dp[i-1]; // single digit decode
            if (twoDigit >= 10 && twoDigit <= 26) dp[i] += dp[i-2]; // two digit decode
        }
        return dp[n];
    }
}
```

## #45 Unique Paths [Medium]

**Companies:** Amazon, Microsoft, Google

### Approach & Key Insight

**Problem: Count paths from top-left to bottom-right of m×n grid moving only right or down.**

**Approach 1: 2D DP**

**- dp[i][j] = number of ways to reach cell (i,j)**

**- dp[i][j] = dp[i-1][j] + dp[i][j-1] (from top + from left)**

**- Base: dp[0][j] = 1, dp[i][0] = 1 (only one way along edges)**

**Approach 2: Combinatorics**

**- Total moves = (m-1) down + (n-1) right = m+n-2 moves**

**- Choose (m-1) down moves from total: C(m+n-2, m-1)**

**Math solution is O(min(m,n)) time — impressive to mention.**

| | |
|---|---|
| ⏱ **Time:** O(m×n) DP or O(min(m,n)) math | 💾 **Space:** O(n) for DP row |

## Java Solution

```java
class Solution {
    // Space-optimized DP: only need one row at a time
    public int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        Arrays.fill(dp, 1); // top row: all 1s

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[j] += dp[j-1]; // dp[j] = from above + from left
            }
        }
        return dp[n-1];
    }

    // Math: C(m+n-2, m-1)
    public int uniquePathsMath(int m, int n) {
        long result = 1;
        // Compute C(m+n-2, min(m-1, n-1)) to avoid overflow
        int k = Math.min(m-1, n-1), total = m + n - 2;
        for (int i = 0; i < k; i++) {
            result = result * (total - i) / (i + 1);
        }
        return (int) result;
    }
}
```

# 🔄 Category XIII — Advanced Linked List & Spiral

Rotate/Spiral Matrix, Reorder List

## #46 Spiral Matrix [Medium]

**Companies:** Amazon, Microsoft, Google

### Approach & Key Insight

**Problem: Return all elements of matrix in spiral order.**

**Approach: Layer-by-layer simulation with 4 boundaries**

**- Maintain top, bottom, left, right pointers**

**- Traverse: right along top → down along right → left along bottom → up along left**

**- After each direction: shrink the corresponding boundary**

**- Stop when top > bottom or left > right**

**Key: Update boundary immediately after each direction to avoid double-counting corners.**

| | |
|---|---|
| ⏱ **Time:** O(m×n) | 💾 **Space:** O(1) excluding output |

### Java Solution

```java
class Solution {

    public List<Integer> spiralOrder(int[][] matrix) {

        List<Integer> result = new ArrayList<>();

        int top = 0, bottom = matrix.length - 1;

        int left = 0, right = matrix[0].length - 1;


        while (top <= bottom && left <= right) {

            // → Right along top row

            for (int c = left; c <= right; c++) result.add(matrix[top][c]);

            top++;

            // ↓ Down along right col

            for (int r = top; r <= bottom; r++) result.add(matrix[r][right]);

            right--;

            // ← Left along bottom row (if still valid)

            if (top <= bottom) {

                for (int c = right; c >= left; c--) result.add(matrix[bottom][c]);

                bottom--;

            }

            // ↑ Up along left col (if still valid)

            if (left <= right) {

                for (int r = bottom; r >= top; r--) result.add(matrix[r][left]);

                left++;

            }

        }

        return result;

    }

}
```

### #47 Rotate Image [Medium]

**Companies:** Amazon, Microsoft

#### Approach & Key Insight

**Problem: Rotate n×n matrix 90° clockwise in-place.**

**Approach: Transpose then Mirror (Reverse each row)**

**- Step 1: Transpose — swap matrix[i][j] with matrix[j][i] (flip along main diagonal)**

**- Step 2: Reverse each row — this achieves the 90° clockwise rotation**

**Mathematical insight:**

**90° clockwise = transpose + reverse rows**

**90° counter-clockwise = transpose + reverse cols (or reverse rows + transpose)**

**180° = reverse rows + reverse cols**

| | |
|---|---|
| ⏱ **Time:** $O(n^2)$ | 💾 **Space:** O(1) in-place |

## Java Solution

```java
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;

        // Step 1: Transpose (flip along main diagonal)
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) { // j starts at i+1 to avoid double-swap
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        // Step 2: Reverse each row
        for (int i = 0; i < n; i++) {
            int left = 0, right = n - 1;
            while (left < right) {
                int temp = matrix[i][left];
                matrix[i][left++] = matrix[i][right];
                matrix[i][right--] = temp;
            }
        }
    }
}
```

## #48 Reorder List [Medium]

**Companies:** Amazon, Google, Microsoft

### Approach & Key Insight

**Problem: Reorder L0→L1→…→Ln into L0→Ln→L1→Ln-1→…**

**Approach: Three Steps**

**1. Find middle (slow/fast pointer)**

**2. Reverse second half**

**3. Merge the two halves alternately**

**This avoids using extra space for array storage.**

**Each step is O(n) and straightforward — combination is the key insight.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(1) |

### Java Solution

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;

        // Step 1: Find middle (slow/fast pointers)
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // slow is now the middle; second half starts at slow.next

        // Step 2: Reverse second half
        ListNode prev = null, curr = slow.next;
        slow.next = null; // cut the list in half
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        // prev is the head of reversed second half

        // Step 3: Merge two halves alternately
        ListNode first = head, second = prev;
        while (second != null) {
            ListNode tmp1 = first.next, tmp2 = second.next;
            first.next = second;
            second.next = tmp1;
            first = tmp1;
            second = tmp2;
        }
    }
}
```

# 🌲 Category XIV — Advanced Tree & Trie

Inorder traversal, LCA, Word Search II (Trie)

## #49 Binary Tree Level Order Traversal [Medium]

**Companies:** Amazon, Google, Meta

### Approach & Key Insight

**Problem: Return node values level by level (left to right).**

**Approach: BFS with queue, process level by level**

**- Use queue; at start of each level, record current queue size = number of nodes in this level**

**- Process exactly that many nodes, adding their children to queue**

**- Collect each level's values into a sublist**

**- This two-loop pattern (outer for levels, inner for nodes in level) is the universal BFS-by-level template.**

| | |
|---|---|
| ⏱ **Time:** O(n) | 💾 **Space:** O(n) queue |

### Java Solution

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size(); // nodes in current level
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < levelSize; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

### #50 Longest Palindromic Subsequence / Palindrome Partitioning II [Hard]

**Companies:** Amazon, Google

#### Approach & Key Insight

**Problem: Find minimum cuts to partition string into palindromes.**

**Approach: DP with palindrome precomputation**

**- isPalin[i][j]: precompute if s[i..j] is palindrome using expand-around-center or 2D DP**

**- dp[i] = min cuts for s[0..i]**

**- If s[0..i] is palindrome: dp[i] = 0 (no cut needed)**

**- Otherwise: dp[i] = min(dp[j] + 1) for all j where s[j+1..i] is palindrome**

**Two-pass DP: first compute all palindrome substrings, then compute minimum cuts.**

| | |
|---|---|
| ⏱ **Time:** O($n^2$) | 💾 **Space:** O($n^2$) palindrome table + O(n) dp |

### Java Solution

```
class Solution {
    public int minCut(String s) {
        int n = s.length();
        boolean[][] isPalin = new boolean[n][n];

        // Precompute all palindrome substrings
        for (int len = 1; len <= n; len++) {
            for (int i = 0; i + len - 1 < n; i++) {
                int j = i + len - 1;
                if (s.charAt(i) == s.charAt(j)) {
                    isPalin[i][j] = (len <= 3) || isPalin[i+1][j-1];
                }
            }
        }

        // dp[i] = minimum cuts for s[0..i]
        int[] dp = new int[n];
        for (int i = 0; i < n; i++) {
            if (isPalin[0][i]) { dp[i] = 0; continue; } // whole prefix is palindrome
            dp[i] = i; // max cuts = i (cut every character)
            for (int j = 1; j <= i; j++) {
                if (isPalin[j][i]) {
                    dp[i] = Math.min(dp[i], dp[j-1] + 1);
                }
            }
        }
        return dp[n-1];
    }
}
```

# Master Strategy — How to Ace LeetCode Interviews at FAANG

| Phase | Action | Details |
|---|---|---|
| Clarify (2 min) | Understand the problem fully | Ask about: constraints (size, sign), edge cases (empty, single element), expected output format, time/space requirements |
| Brute Force (2 min) | State the naive solution | Always describe $O(n^2)$ or recursive solution first. Shows understanding. Establishes baseline to optimize from. |
| Optimize (5 min) | Walk through optimizations | Think out loud: 'Can I sort first? Use a HashMap? Sliding window? DP?' Draw examples on whiteboard. |
| Code (15 min) | Write clean, readable code | Use meaningful variable names. Add comments for non-obvious logic. Handle edge cases explicitly. |
| Test (3 min) | Verify with examples | Walk through 2-3 examples: normal case, edge case (empty/single), and a tricky case. Trace through your code. |
| Complexity (2 min) | State time and space | Always analyze both. Justify why. Mention if space can be reduced (e.g., rolling array for DP). |

## Pattern-to-Problem Mapping

| If you see... | Think... | Example Problems |
|---|---|---|
| Array + find pair/triplet summing to X | Two Pointers or HashMap | Two Sum, 3Sum, 4Sum |
| Substring / subarray with constraint | Sliding Window | Min Window, Longest Substring |
| k-th largest/smallest, stream | Heap (PriorityQueue) | Kth Largest, Median Stream, Merge K Lists |
| Shortest path / least steps | BFS (unweighted) | Word Ladder, Shortest Path in Grid |
| All combinations / permutations | Backtracking | N-Queens, Subsets, Permutations |
| Overlapping subproblems, memoization | Dynamic Programming | Coin Change, Edit Distance, LIS |
| Sorted array + O(log n) required | Binary Search | Rotated Array, Kth in Two Arrays |
| Next greater / smaller element | Monotonic Stack | Histogram, Daily Temperatures |
| Graph traversal, connected components | DFS/BFS + Visited Set | Islands, Clone Graph |
| Build order, course prerequisites | Topological Sort (Kahn's) | Course Schedule, Alien Dict |
| Prefix matching, autocomplete | Trie | Word Search II, Implement Trie |
| Range sum queries | Prefix Sum Array | Subarray Sum Equals K |
| Interval scheduling, no overlap | Greedy + Sort by end time | Meeting Rooms, Activity Selection |
| Tree traversal + BST properties | Inorder = sorted; use DFS bounds | Validate BST, Kth Smallest |